

**TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN**



Giám sát năng lượng tiêu thụ với ESP32

TÊN LỚP HỌC PHẦN : Phát triển ứng dụng IoT

MÃ HỌC PHẦN: 2025-2026.2.TIN4024.001

GIẢNG VIÊN HƯỚNG DẪN: Võ Việt Dũng

HUẾ, THÁNG 4 NĂM 2026

DANH MỤC VIẾT TẮT

Từ viết tắt	Nghĩa
AC	(Alternating Current): Dòng điện xoay chiều.
ADC	(Analog-to-Digital Converter): Bộ chuyển đổi tín hiệu tương tự sang kỹ thuật số.
API	(Application Programming Interface): Giao diện lập trình ứng dụng.
DC	(Direct Current): Dòng điện một chiều.
ESP32	Vi điều khiển hệ thống trên chip có tích hợp Wi-Fi và Bluetooth.
IoT	(Internet of Things): Internet vạn vật.
JSON	(JavaScript Object Notation): Định dạng dữ liệu văn bản.

MỤC LỤC

I.Mở đầu	1
1.Giới thiệu	1
2.Mục Đích	1
3.Mục tiêu đề tài	2
II.Nội Dung	3
1.Giới thiệu về ESP32.....	3
2.Cảm biến ACS712	4
Nguyên lý hoạt động.....	5
3.Cơ chế hoạt động	5
III.Thiết kế kiến trúc hệ thống	7
1. Kiến trúc hệ thống IoT 3 lớp.....	7
2. Kỹ thuật lập trình và Tối ưu hóa thuật toán.....	7
3. Chương trình ESP32 mô phỏng cảm biến dòng điện ACS712.....	8
-Kết nối tới blynk.....	9
-Kết nối tới telegram:	11
IV. PHẦN KẾT LUẬN/NHẬN XÉT/ĐÁNH GIÁ/KIẾN NGHỊ	12
1. Kết quả đạt được:	12
2. Hướng phát triển trong tương lai:	12
V.Tài liệu tham khảo:	13
VI. PHỤ LỤC	14
1. Mã nguồn chương trình điều khiển trung tâm (main.cpp).....	14
2. Tập cấu hình kiến trúc phần cứng (diagram.json)	18
VII. Phiếu đánh giá tiêu luận:	20

I.Mở đầu

1.Giới thiệu

Trong kỷ nguyên Công nghiệp 4.0, khái niệm nhà thông minh (Smart Home) và lưới điện thông minh (Smart Grid) đang dần trở thành tiêu chuẩn tất yếu. Đi kèm với sự phát triển đó, nhu cầu sử dụng năng lượng điện của con người ngày càng tăng cao, đặt ra những thách thức to lớn về việc quản lý, tiết kiệm năng lượng và đảm bảo an toàn lưới điện. Việc giám sát năng lượng tiêu thụ không chỉ mang ý nghĩa thiết thực trong việc tối ưu hóa chi phí vận hành cho các hộ gia đình hay doanh nghiệp, mà còn góp phần quan trọng vào chiến lược bảo vệ môi trường toàn cầu, giảm thiểu phát thải khí nhà kính.

Xuất phát từ thực tiễn đó, dự án "Giám sát năng lượng tiêu thụ với ESP32" được nghiên cứu và xây dựng nhằm tạo ra một giải pháp IoT toàn diện. Hệ thống tận dụng sức mạnh xử lý mạnh mẽ của vi điều khiển ESP32 để đo lường công suất tiêu thụ, đồng bộ hóa dữ liệu lên nền tảng đám mây (Cloud), đồng thời cung cấp cơ chế cảnh báo tự động đa kênh, giúp người dùng kiểm soát hệ thống điện mọi lúc, mọi nơi một cách trực quan nhất.

2.Mục Đích

Giám sát liên tục, đồng bộ thời gian thực: Đo lường và theo dõi giá trị dòng điện, tính toán công suất tiêu thụ liên tục. Dữ liệu được đồng bộ xuyên suốt trên đa nền tảng thông qua hệ sinh thái Blynk (Blynk Web Dashboard dành cho hệ thống quản lý trung tâm và Mobile App dành cho người dùng cá nhân).

Phát hiện sự cố và phản hồi nhanh: Tự động nhận diện các trường hợp bất thường như ngắn mạch, thiết bị hoạt động quá công suất thông qua việc giám sát các ngưỡng dòng điện an toàn đã được thiết lập.

Hệ thống cảnh báo đa kênh thông minh (Anti-Spam): Khắc phục nhược điểm của các hệ thống cảnh báo truyền thống, dự án tự động gửi tin nhắn báo động qua Telegram Bot và cảnh báo Notification trên ứng dụng. Đặc biệt, hệ thống tích hợp thuật toán thời gian chờ (Cooldown) để không gây nghẽn mạng hay spam tin nhắn rác đến người dùng.

Trực quan hóa trạng thái phần cứng: Sử dụng cụm đèn LED vật lý để hiển thị trạng thái hoạt động (sáng ổn định khi bình thường, chớp nháy liên tục khi hệ thống gặp lỗi quá tải).

3. Mục tiêu đề tài

Khai thác cấu trúc phần cứng và lập trình vi điều khiển ESP32 làm bộ xử lý trung tâm điều phối toàn bộ các tác vụ đa luồng.

Tìm hiểu cơ sở lý thuyết và mô phỏng nguyên lý làm việc của cảm biến Hall ACS712 trong việc đo lường dòng điện AC/DC.

Xây dựng luồng dữ liệu (Datastreams) ổn định, thiết lập giao thức kết nối API bảo mật giữa phần cứng và nền tảng đám mây (Blynk IoT, Telegram Bot).

Triển khai mô phỏng, lập trình và kiểm thử mạch điện thực tế hoàn chỉnh trên nền tảng Wokwi trước khi đưa vào sản xuất thực tế.

II. Nội Dung

1. Giới thiệu về ESP32

ESP32 là một hệ thống vi điều khiển trên chip (SoC) hiệu năng cao, giá rẻ được phát triển bởi Espressif Systems [3]. Vượt trội hơn hẳn so với người tiền nhiệm ESP8266 hay các dòng Arduino truyền thống, ESP32 được thiết kế đặc biệt cho các ứng dụng IoT phức tạp nhờ những đặc điểm kỹ thuật sau:

- **Vi xử lý mạnh mẽ:** Trang bị chip Xtensa Dual-Core 32-bit LX6 hoạt động ở xung nhịp lên đến 240MHz, đi kèm 520KB SRAM, cho phép xử lý các thuật toán tính toán công suất phức tạp mà không bị trễ (lag).
- **Tích hợp truyền thông:** Hỗ trợ đồng thời Wi-Fi chuẩn 802.11 b/g/n và Bluetooth v4.2/BLE, giúp mạch dễ dàng kết nối với Router mạng gia đình để đẩy dữ liệu lên Internet.
- **Bảo mật phần cứng:** Cung cấp tài nguyên phần cứng để mã hóa chuẩn bảo mật SSL/TLS. Đây là yếu tố mang tính quyết định giúp ESP32 có thể giao tiếp an toàn với các API HTTPS của bên thứ ba như Telegram.
- **Hệ thống ADC độ phân giải cao:** ESP32 sở hữu các bộ chuyển đổi tương tự - số (ADC) 12-bit, cho phép chia dải điện áp đo được thành 4096 mức, giúp việc đọc tín hiệu từ cảm biến đo dòng điện trở nên cực kỳ chính xác.



2.Cảm biến ACS712

Cảm biến ACS712 hoạt động dựa trên hiệu ứng Hall để đo dòng điện AC/DC [1]. Trị số trả về là điện áp Analog tuyến tính theo cường độ dòng điện, giúp vi điều khiển dễ dàng quy đổi sang giá trị Ampe [1]. Thông số kỹ thuật nổi bật [1]:

- Thời gian chuyển đổi: $5\mu\text{s}$, độ nhạy đầu ra: $63 - 190 \text{ mV/A}$.
- Mức điện áp an toàn cách ly tối đa: 2100V .

Đường tín hiệu analog có độ nhiễu thấp	
Thời gian chuyển đổi	$5\mu\text{s}$

Điện trở trong	1.2mΩ
Sử dụng nguồn điện	5V
Độ nhạy đầu ra	63 – 190 mV/A
Nhiệt độ hoạt động	-40 – 85 0C
Điện áp cách ly tối đa	2100V (RMS)

Nguyên lý hoạt động

- Hiệu ứng Hall: ACS712 sử dụng hiệu ứng Hall để đo từ trường sinh ra bởi dòng điện đi qua dây dẫn bên trong cảm biến.
- Điện áp đầu ra:
 - Khi không có dòng điện (0A), điện áp đầu ra là 2.5V (điểm giữa của nguồn 5V).
 - Khi có dòng điện dương, điện áp tăng (>2.5V). o Khi có dòng điện âm, điện áp giảm (<2.5V).

3.Cơ chế hoạt động

Hệ thống đo dòng điện tiêu thụ qua thiết bị điện nhờ cảm biến ACS712, xử lý dữ liệu trên ESP32, và gửi thông tin lên ứng dụng Blynk qua kết nối Wi-Fi. Blynk sẽ hiển thị dữ liệu trực quan bằng các widget như Value Display, Graph, hoặc LED giúp người dùng giám sát dòng điện/công suất theo thời gian thực ngay trên điện thoại.

Quy trình hoạt động:

Bước 1: Đo dòng điện

Dòng điện tiêu thụ chạy qua cảm biến ACS712.

Cảm biến xuất ra một điện áp analog tỉ lệ thuận với dòng điện.

Bước 2: Đọc tín hiệu từ cảm biến

ESP32 đọc điện áp từ ACS712 qua chân ADC (Analog-to-Digital Converter).

Tín hiệu được xử lý để tính ra dòng điện thực tế theo công thức tuyến tính:

$$I = \frac{V_{\text{out}} - V_{\text{zero}}}{\text{Hệ số cảm biến}}$$

Trong đó:

- $V_{\text{zero}} \approx 2.5V$
- Hệ số cảm biến (như 185mV/A đối với bản 5A)

Bước 3: Tính công suất tiêu thụ (tùy chọn)

Nếu muốn hiển thị công suất:

$$P=U \times I$$

Trong đó:

U là điện áp tải (giả định là 220V)

I là dòng điện đo được

Bước 4: Gửi dữ liệu lên Blynk

ESP32 kết nối Wi-Fi và sử dụng Blynk Virtual Pin (ví dụ V0, V1) để gửi dữ liệu dòng/công suất.

Người dùng mở app Blynk để theo dõi dữ liệu hiển thị theo thời gian thực.

III. Thiết kế kiến trúc hệ thống

1. Kiến trúc hệ thống IoT 3 lớp

Hệ thống được thiết kế chặt chẽ theo mô hình tiêu chuẩn của một hệ sinh thái IoT:

- **Lớp thu thập (Perception Layer):** Chịu trách nhiệm cảm nhận môi trường vật lý. Cảm biến ACS712 liên tục đo mức tiêu thụ điện năng từ tải và truyền tín hiệu Analog về chân GPIO của vi điều khiển.
- **Lớp mạng và Xử lý (Network & Processing Layer):** ESP32 đóng vai trò là "bộ não". Nó chuyển đổi tín hiệu ADC sang cường độ dòng điện, nhân với điện áp lưới giả định (220V) để suy ra công suất tiêu thụ $P = U \times I$. Đồng thời, lớp này chịu trách nhiệm đóng gói dữ liệu và mở kết nối Wi-Fi TCP/IP.
- **Lớp Ứng dụng (Application Layer):** Bao gồm giao diện người dùng và hệ thống cảnh báo.
 - *Blynk Cloud:* Dữ liệu được đẩy lên các kênh ảo (Virtual Pins: V1 cho Dòng điện, V2 cho Công suất). Nền tảng này tự động vẽ biểu đồ và hiển thị thông số đồng bộ trên cả máy tính và điện thoại thông minh [2].
 - *Telegram Bot:* Nhận lệnh API HTTPS từ ESP32 để tự động đẩy tin nhắn báo động chi tiết đến tài khoản người quản lý [4].

2. Kỹ thuật lập trình và Tối ưu hóa thuật toán

Do giới hạn của môi trường mô phỏng Wokwi không có module ACS712 vật lý, dự án thiết kế một hàm mô phỏng sử dụng bộ sinh số ngẫu nhiên, tạo ra các giá trị dòng điện

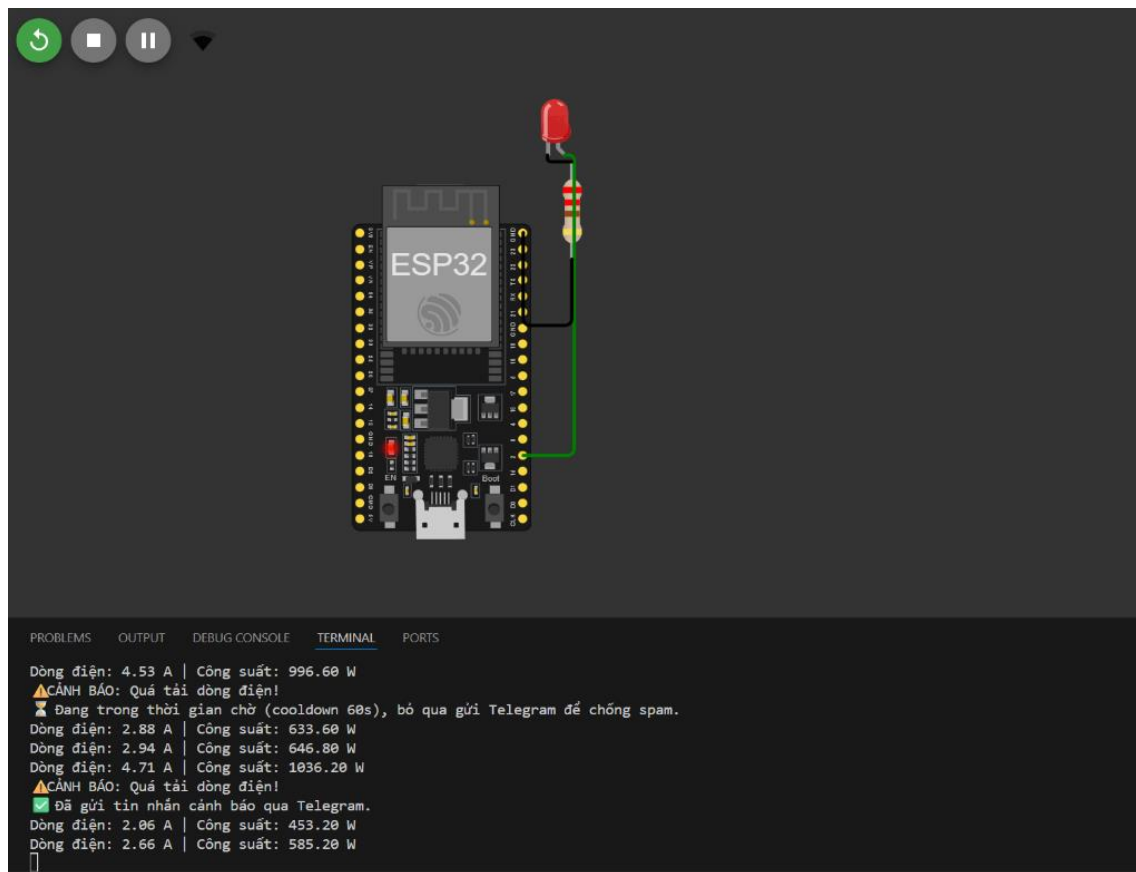
biến thiên từ 0A đến 5A để giả lập lưới điện [5]. Điểm sáng giá của hệ thống nằm ở hai thuật toán cốt lõi:

- **Kỹ thuật Non-blocking Delay (Đa nhiệm không đồng bộ):** Thay vì sử dụng hàm delay() truyền thống làm đóng băng toàn bộ hệ thống, mã nguồn sử dụng hàm millis() để tính toán độ trễ. Kỹ thuật này giúp ESP32 vẫn duy trì kết nối Wi-Fi liên tục, phản hồi lệnh từ người dùng ngay lập tức trong khi chờ đến chu kỳ 5 giây để cập nhật dữ liệu cảm biến mới.
- **Thuật toán chống Spam Telegram (Cooldown Mechanism):** Khi hệ thống điện chập chờn, dòng điện có thể liên tục dao động quanh mức cảnh báo (ví dụ: 4.1A rồi 3.9A rồi 4.2A). Nếu không có cơ chế kiểm soát, bot Telegram sẽ gửi hàng chục tin nhắn mỗi giây, gây treo ứng dụng và cạn kiệt tài nguyên mạng. Thuật toán giải quyết bằng cách sử dụng cờ isFirstAlert. Khi quá tải xảy ra lần đầu, tin nhắn được gửi ngay lập tức. Sau đó, hệ thống sẽ bị khóa tính năng gửi tin nhắn trong đúng 60 giây tiếp theo, bất chấp việc lưới điện vẫn đang lỗi. Trong thời gian khóa này, đèn LED vật lý tại hiện trường vẫn chớp nháy liên tục để cảnh báo bằng thị giác.

3. Chương trình ESP32 mô phỏng cảm biến dòng điện ACS712

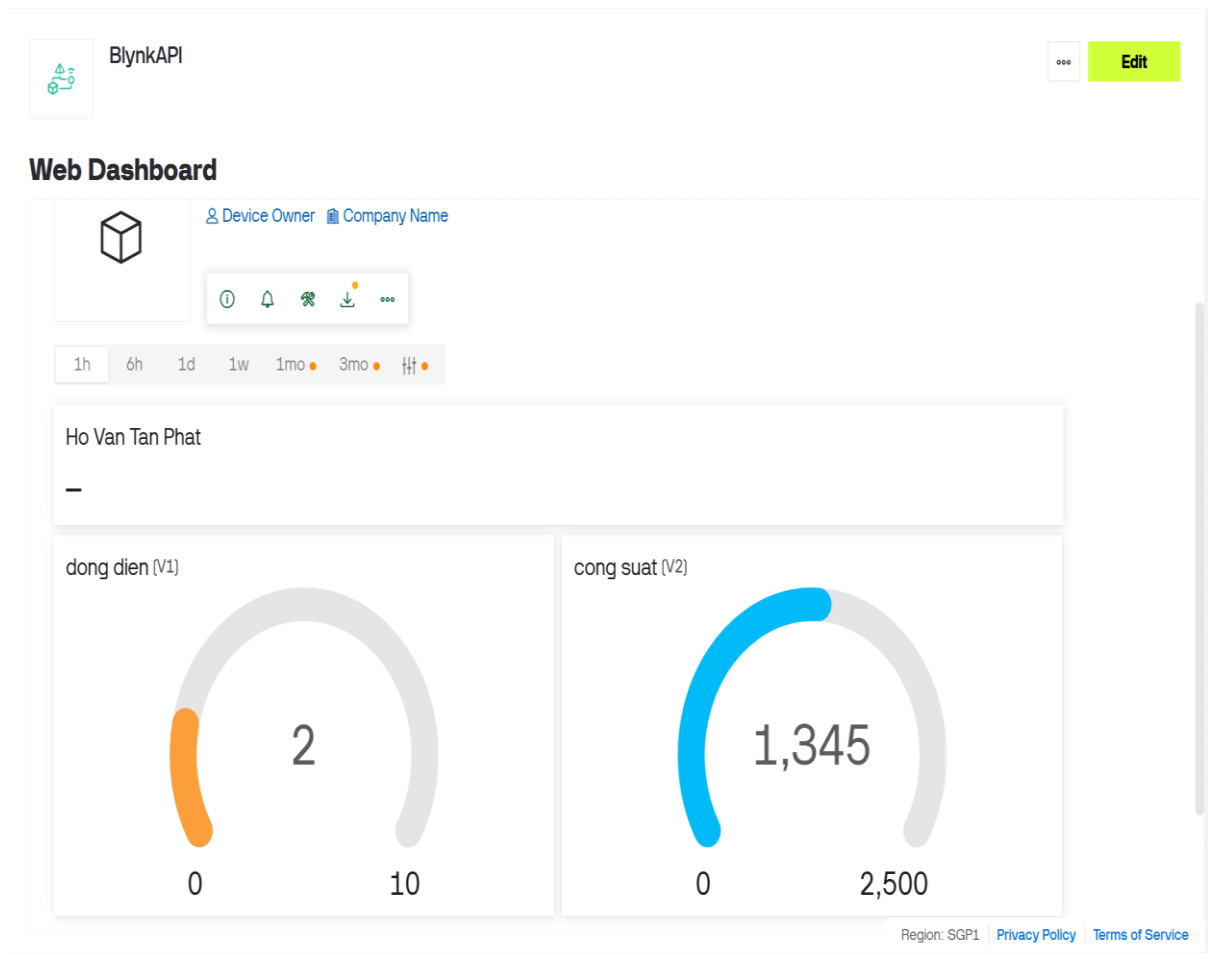
Vì Wokwi không hỗ trợ cảm biến ACS712, mã sử dụng giá trị ngẫu nhiên để giả lập dòng điện:

- Tính công suất tiêu thụ.
- Hiển thị thời gian thực và gửi cảnh báo qua Blynk.
- Lưu trữ lịch sử dữ liệu trên ThingSpeak.
- Điều khiển LED để giả lập trạng thái.
- Mô phỏng cảm biến ACS712.

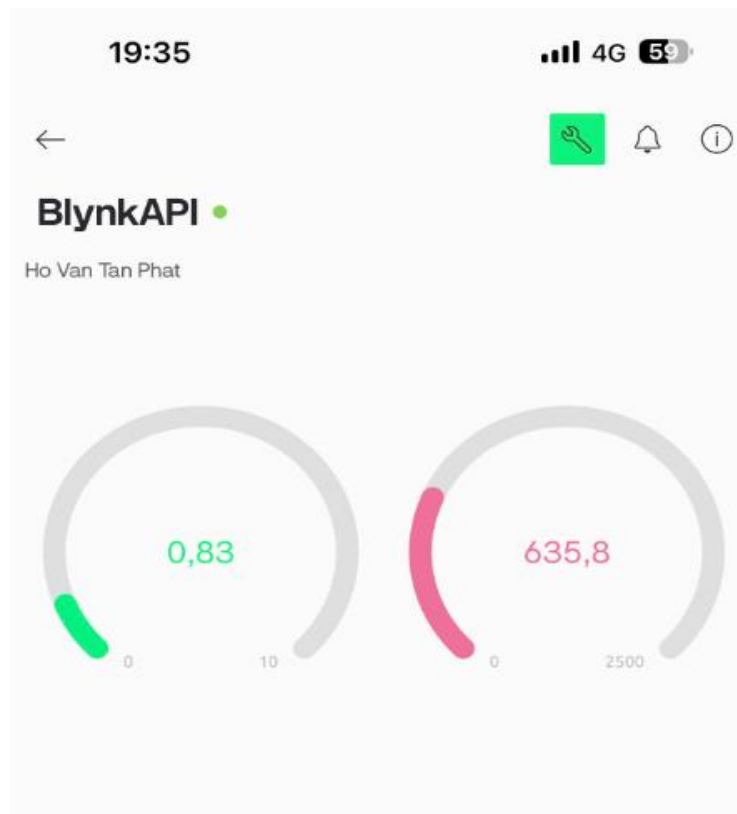


-Kết nối tới blynk

Có thể xem cường độ dòng điện và công suất thông qua Blynk đồng thời nhận cảnh báo nếu dòng điện vượt ngưỡng an toàn tối thiết bị của bạn.

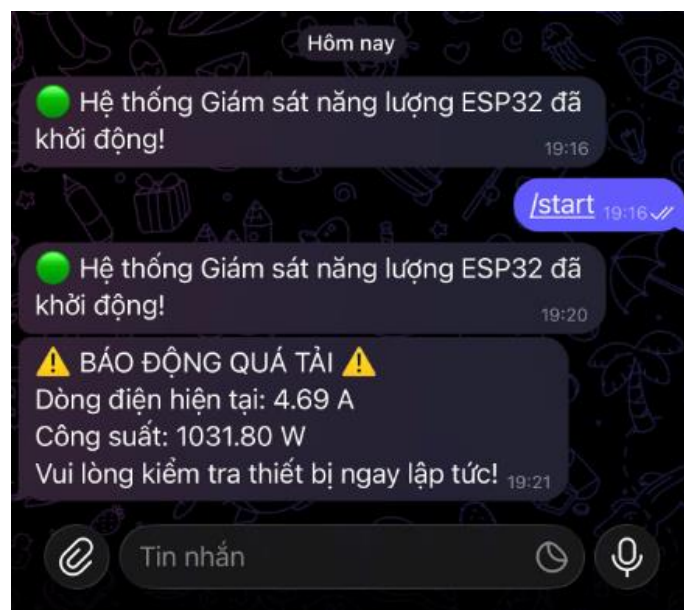


Hình 1: Giao diện giám sát thời gian thực trên Blynk Web App



Hình 2: Giao diện giám sát thời gian thực trên Blynk Mobile App

-Kết nối tới telegram:



Hình 3: Giao diện giám sát thời gian thực trên Telegram

IV. PHẦN KẾT LUẬN/NHẬN XÉT/ĐÁNH GIÁ/KIẾN NGHỊ

1. Kết quả đạt được: Dự án đã triển khai thành công mô hình IoT ứng dụng trong quản lý năng lượng trên Wokwi. ESP32 đã hoàn thành xuất sắc vai trò xử lý trung tâm khi vừa duy trì luồng dữ liệu thời gian thực mượt mà lên hệ sinh thái Blynk (Web & Mobile), vừa xử lý trơn tru các tác vụ giao tiếp bảo mật với máy chủ Telegram. Thuật toán Non-blocking giúp mạch không bị treo trong quá trình chạy, các thông báo sự cố hoạt động chính xác theo kịch bản đề ra.

2. Hướng phát triển trong tương lai:

- Đưa mô hình vào thực tế bằng cách thi công mạch in (PCB) với cảm biến ACS712 và ESP32 thật.
- Tích hợp thêm cảm biến đo điện áp ZMPT101B để tính toán công suất thực tế thay vì sử dụng hằng số giả định 220V.
- Bổ sung tính năng điều khiển đóng/ngắt Relay hai chiều từ xa qua Blynk hoặc lệnh chat Telegram để tự động ngắt nguồn thiết bị khi phát hiện quá tải kéo dài.

V.Tài liệu tham khảo:

- [1] Allegro Microsystems (2025). ACS712 Datasheet. <https://www.allegromicro.com/en/products/sense/current-sensor-ics/acs712>.
- [2] Blynk Documentation (2025). Blynk Official Documentation. <https://docs.blynk.io/>.
- [3] Espressif Systems (2025). ESP32 Technical Reference Manual. <https://www.espressif.com/en/support/documents/technical-documents>.
- [4] Telegram API Documentation (2025). Telegram Bot API. <https://core.telegram.org/bots/api>.
- [5] Wokwi Documentation (2025). Wokwi Simulator Guide. <https://docs.wokwi.com/>.

VI. PHỤ LỤC

Phân phụ lục này cung cấp toàn bộ mã nguồn và các tệp tin cấu hình môi trường cần thiết để tái tạo, kiểm thử dự án trên nền tảng mô phỏng Wokwi cũng như phần mềm lập trình chuyên dụng VS Code (PlatformIO).

1. Mã nguồn chương trình điều khiển trung tâm (main.cpp)

Tệp tin chứa toàn bộ logic xử lý đo lường, kết nối mạng, giao tiếp với nền tảng Blynk Cloud và gửi cảnh báo khẩn cấp thông qua API của Telegram. Đoạn mã đã được tối ưu hóa bằng phương pháp Non-blocking.

```
/*  
 * Đồ án: Giám sát năng lượng tiêu thụ với ESP32  
 * Chức năng: Giả lập dòng điện (ACS712), tính công suất, giám sát qua Blynk (Web &  
 Mobile) và cảnh báo Telegram  
 */  
  
#define BLYNK_TEMPLATE_ID "YOUR_TEMPLATE_ID"    // Thay bằng ID  
Template của bạn  
#define BLYNK_DEVICE_NAME "Energy Monitor"  
#define BLYNK_AUTH_TOKEN "YOUR_AUTH_TOKEN"    // Thay bằng Token  
thiết bị của bạn  
#define BLYNK_PRINT Serial  
  
#include <WiFi.h>  
#include <WiFiClientSecure.h>  
#include <BlynkSimpleEsp32.h>  
#include <UniversalTelegramBot.h>
```

```

#include <ArduinoJson.h>

// --- CẤU HÌNH WIFI (Dùng mạng mặc định của hệ thống Wokwi) ---
char ssid[] = "Wokwi-GUEST";
char pass[] = "";

// --- CẤU HÌNH TELEGRAM BOT ---
#define BOT_TOKEN "ĐIỀN_TOKEN_TELEGRAM_CỦA_BẠN_VÀO_ĐÂY"
#define CHAT_ID "ĐIỀN_CHAT_ID_CỦA_BẠN_VÀO_ĐÂY"

WiFiClientSecure secured_client;
UniversalTelegramBot bot(BOT_TOKEN, secured_client);

// --- CẤU HÌNH GIAO TIẾP PHẦN CỨNG ---
const int ledPin = 2;    // Chân điều khiển tín hiệu đèn LED cảnh báo
const float VOLTAGE = 220.0; // Điện áp lưới giả định để tính toán công suất

// --- BIẾN TOÀN CỤC QUẢN LÝ THỜI GIAN (NON-BLOCKING) ---
unsigned long lastUpdateTime = 0;
const unsigned long updateInterval = 5000; // Chu kỳ cập nhật dữ liệu: 5 giây/lần

unsigned long lastTelegramAlert = 0;
const unsigned long telegramCooldown = 60000; // Chu kỳ chống spam Telegram: 60 giây
bool isFirstAlert = true; // Cờ trạng thái ưu tiên gửi cảnh báo đầu tiên

void setup() {
    Serial.begin(115200);

```

```

pinMode(ledPin, OUTPUT);
digitalWrite(ledPin, LOW);

Serial.println("Hệ thống đang khởi tạo kết nối WiFi...");

// Thiết lập chứng chỉ bảo mật cho Telegram (bỏ qua kiểm tra xác thực SSL trên Wokwi)
secured_client.setInsecure();

// Khởi tạo kết nối đến máy chủ Blynk
Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

Serial.println("\nKhởi động hệ thống Giám sát năng lượng thành công!");

// Gửi tin nhắn kiểm tra trạng thái hoạt động lên nhóm Telegram
bot.sendMessage(CHAT_ID, "🌀 Hệ thống Giám sát năng lượng ESP32 đã khởi động
và sẵn sàng hoạt động!", "");
}

void loop() {
// Hàm duy trì luồng kết nối liên tục với máy chủ Blynk
Blynk.run();

// Kiểm tra thời gian để thực thi các tác vụ định kỳ (Non-blocking)
if (millis() - lastUpdateTime >= updateInterval) {
    lastUpdateTime = millis();

// 1. Giả lập quá trình đọc cảm biến dòng điện ACS712 (Biến thiên từ 0A đến 5A)
float current = random(0, 500) / 100.0;

```

```

// 2. Tính toán công suất tiêu thụ hiện tại ( $P = U * I$ )
float power = current * VOLTAGE;

// 3. Hiển thị thông số ra màn hình Terminal (\r\n để chống trôi ký tự)
Serial.printf("Dòng điện thực tế: %.2f A | Công suất tiêu thụ: %.2f W\r\n", current,
power);

// 4. Đồng bộ hóa dữ liệu lên nền tảng đám mây Blynk Cloud
// Ghi dữ liệu dòng điện vào kênh ảo V1 và công suất vào kênh ảo V2
Blynk.virtualWrite(V1, current);
Blynk.virtualWrite(V2, power);

// 5. Xử lý kịch bản lỗi hệ thống: Cảnh báo quá tải khi dòng điện vượt ngưỡng 4.0A
if (current > 4.0) {
    Serial.println("⚠ CẢNH BÁO MỨC ĐỘ CAO: Lưới điện đang quá tải!");
    digitalWrite(ledPin, HIGH); // Kích hoạt tín hiệu LED báo động

    // Đẩy sự kiện cảnh báo (Event) lên hệ thống ghi nhận của Blynk
    Blynk.logEvent("current_warning", "Cảnh báo khẩn: Dòng điện vượt ngưỡng an
toàn 4A!");

    // Khởi chạy thuật toán gửi tin nhắn Telegram chống Spam
    if (isFirstAlert || millis() - lastTelegramAlert >= telegramCooldown) {
        String alertMsg = "⚠ **BÁO ĐỘNG QUÁ TẢI LƯỚI ĐIỆN** ⚠\n";
        alertMsg += "Trạng thái dòng điện hiện tại: " + String(current, 2) + " A\n";
        alertMsg += "Công suất tiêu thụ tức thời: " + String(power, 2) + " W\n";
    }
}

```

```
    alertMsg += "Yêu cầu kỹ thuật viên kiểm tra thiết bị ngay lập tức để phòng chống  
cháy nổ!";
```

```
    bot.sendMessage(CHAT_ID, alertMsg, "Markdown");
```

```
    lastTelegramAlert = millis(); // Đặt lại bộ đếm thời gian chờ
```

```
    isFirstAlert = false; // Tắt cờ ưu tiên
```

```
    Serial.println("☑ Tác vụ hoàn thành: Đã gửi thông báo khẩn cấp qua Telegram.");
```

```
  } else {
```

```
    Serial.println("⌚ Thông báo: Đang trong thời gian chờ (cooldown 60s), tạm hoãn  
gửi tin nhắn Telegram.");
```

```
  }
```

```
  } else {
```

```
    digitalWrite(ledPin, LOW); // Tắt đèn LED nếu thông số trở về mức an toàn
```

```
  }
```

```
}
```

```
}
```

2. Tập cấu hình kiến trúc phần cứng (diagram.json)

Tập mã định dạng JSON này được sử dụng trên nền tảng Wokwi để khai báo vị trí, thông số linh kiện và quy hoạch các đường dây nối mạch điện từ ESP32 đến các linh kiện ngoại vi.

```
{
```

```
  "version": 1,
```

```
  "author": "Thinh",
```

```
  "editor": "wokwi",
```

```
  "parts": [
```

```
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": 0, "attrs": {} },
```

```
    {
```

```

    "type": "wokwi-led",
    "id": "led-status",
    "top": -60,
    "left": 100,
    "attrs": { "color": "red" }
  },
  {
    "type": "wokwi-resistor",
    "id": "r1",
    "top": 10,
    "left": 100,
    "rotate": 90,
    "attrs": { "value": "220" }
  }
],
"connections": [
  [ "esp:TX", "$serialMonitor:RX", "", [] ],
  [ "esp:RX", "$serialMonitor:TX", "", [] ],
  [ "esp:2", "led-status:A", "green", [ "h30", "v-80" ] ],
  [ "led-status:C", "r1:1", "black", [ "v20" ] ],
  [ "r1:2", "esp:GND.2", "black", [ "v40", "h-40" ] ]
]
}

```

VII. Phiếu đánh giá tiểu luận:

[Mẫu TL2]

TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA.....

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

PHIẾU ĐÁNH GIÁ TIỂU LUẬN

Học kỳ Năm học ...-...

	SỐ PHÁCH:.....
Cán bộ chấm thi 1	Cán bộ chấm thi 2
Nhận xét:	Nhận xét:
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
Điểm đánh giá của CBChT1:	Điểm đánh giá của CBChT2:
Bằng số:	Bằng số:
.....	
Bằng chữ:	Bằng chữ:
.....	

Điểm kết luận: Bảng số..... Bảng chữ:.....

Thừa Thiên Huế, ngày tháng năm

20...

CBC_hT1

(Ký và ghi rõ họ tên)

CBC_hT2

(Ký và ghi rõ họ tên)

